



Detección de esteganografía a través de un modelo de aprendizaje automático

Miguel Ángel Tena García (A01709653)
Tecnológico de Monterrey, Campus Querétaro
29 de abril de 2025

1. Introducción

A lo largo del tiempo, se han buscado diferentes formas de esconder información de manera creativa, muy secreta y difícil de encontrar, desde métodos clásicos y mecánicos como el uso de tinta invisible, o las técnicas de microfilmación en textos durante la segunda guerra mundial (1) hasta las técnicas digitales más modernas como lo puede ser la esteganografía de diferentes dominios. La esteganografía digital aprovecha las características de archivos multimedia (imágenes, audio y video) para incrustar información de forma imperceptible, con el objetivo de generar distintos métodos de comunicación discreta.

Tradicionalmente el estegoanálisis, se llevaba a cabo en dos etapas, primero se diseñaba manualmente complejos extractores de características, después entrenando un clasificador para discriminar entre las imágenes cover y stego (2). Estos enfoques suelen ser muy eficaces, pero su dependencia a la correcta extracción de las características y a parámetros hechos a mano limita mucho su adaptabilidad a variantes emergentes de esteganografía.

En los últimos años, con el auge de la inteligencia artificial, las redes neuronales convolucionales, han permitido enfocar de forma end-to-end, esto nos ayuda a unificar la extracción de características y la clasificación en un único modelo entrenable al emplear capas iniciales de preprocesado, como lo pueden ser filtros de alta frecuencia u operaciones de pooling (3), las CNN's pueden aprender directamente de la señal residual (4). Estudios recientes demuestran que estas arquitecturas superan por mucho a los métodos que se basan en modelos ricos (Rich Models) como en dominios espaciales (5).

En este trabajo se propone un pipeline de esteganálisis que combina:

- 1.- Un bloque fijado de filtros de alta frecuencia.
- 2.- Operación de Pooling para conservar información de bajo nivel.
- 3.- Una CNN backbone refinada mediante transfer-learning usando VGG16 pretrained.
- 4.- Capas densas finales optimizadas para la clasificación binaria.

Para el desarrollo de este proyecto nos basamos y utilizamos el dataset de ALASKA2 un conjunto de 75,000 imágenes 512x512 de formato '**.JPG**', una competencia de estegoanálisis, lo que nos ayudaría a mejorar frente a métodos clásicos y a CNN's entrenadas desde cero.

2.- Detección de esteganografía utilizando CNN's.

Aquí se propone un esquema de detección de esteganografía en imágenes basado en tres componentes principales:

- Preprocesamiento con banco de filtros (kernels) para resaltar sutilezas de la esteganografía
- Extracción de características mediante una red convolucional (CNN) inspirada en los diseños de SRNet y YeNet
- Transfer-Learning sobre VGG16, preentrenada en ImageNet para aprovechar su entrenamiento y después usar la implementación como clasificador.

2.1.- Preprocesamiento:

Para fortalecer y resaltar las señales residuales de la esteganografía, empleé un banco estático de kernels de alta frecuencia inspirados en los modelos espaciales ricos (SRM) [6], lo que ayuda a que se capturen bordes, diagonales y cruces, implementándose como una capa convolucional, por cuestiones de tiempo, y capacidad de cómputo solo un pequeño banco de 3 kernels de 5x5 en lugar del planteado de 30 y se implementa como capa con pesos fijos, de modo que en las salidas se destacan las perturbaciones incrustadas.

Este paso inicial hace mucho más fácil la separación entre las clases **cover** y **stego** antes de la etapa convolucional, lo que favorece reduciendo el riesgo de overfitting sobre ruido de baja frecuencia.

Además, también por el límite de poder de computó, fue necesario procesar las imágenes, haciendo un reescalado de un tamaño original de 512x512 píxeles a 256x256, esto debido a que también en muchos artículos científicos, se utiliza esta medida de reescalado sin presentar pérdidas en los detalles de la esteganografía.

2.2.- Arquitectura CNN entrenada

Aquí, partimos de las primeras investigaciones de CNN's para estegoanálisis, donde el diseño del modelo secuencial fue compuesto por:

- Capa de procesamiento SRM + L2-Pooling (en lugar de maxpool para poder preservar la energía del ruido residual [7])
- Tres Bloques Convolucionales
- GlobalAveragePooling (extraer características) y clasificador denso de 128 neuronas con una capa de salida **sigmoide** lo que ayuda al aprendizaje no lineal.

2.3.- Transfer learning con VGG16 pre-entrenada en ImageNet

Se buscó un enfoque híbrido propuesto por el profesor Benjamín, utilizando la arquitectura VGG16, lo que ayuda a mejorar la generalización y acelerar el entrenamiento con bases de tamaño moderado. Al no utilizar el head, buscamos solamente utilizar el extractor de características (los bloques convolucionales). Después congelamos las primeras capas para conservar los patrones de bajo nivel que ya se aprendieron en ImageNet. Finalmente, la salida de VGG16 la pasamos por nuestro banco de kernels (filtro alta frecuencia) y al clasificador denso personalizado, adaptado a la naturaleza binaria del estegoanálisis.

2.4.-Segregación de dataset de entrenamiento y evaluación

El dataset de 75,000 imágenes se dividió en 70% para entrenamiento, 20% para validación y 10% para prueba, dividido por clases. Además, se separan 5 imágenes de cover y de JMiPOD, para las hot test, que son las últimas 5. Utilizamos una optimización “**Adam**” con `binary_crossentropy` y callbacks para guardar los mejores modelos monitoreando el `val_accuracy`.

Finalmente utilizamos métricas importantes como la mencionada anteriormente para medir el desempeño de los modelos a evaluarlos como la matriz de confusión y el reporte de clasificación como precisión, recall, F1-Score.

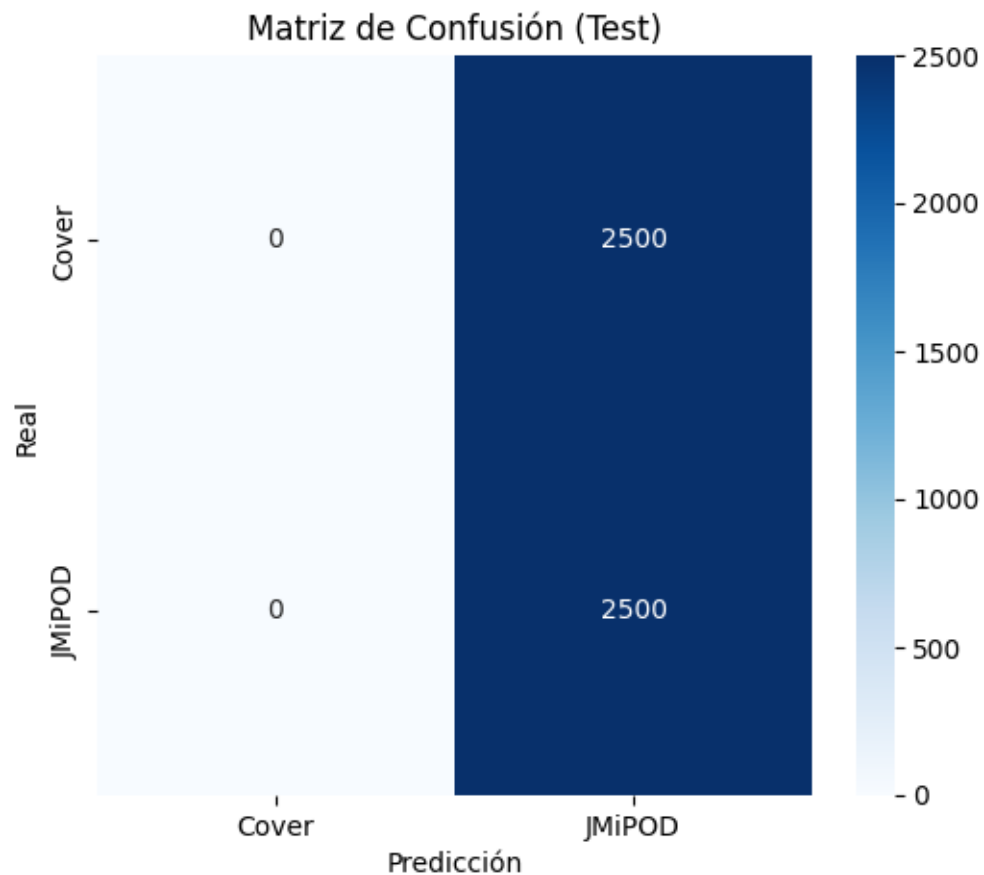
3.- Experimentación y resultados

Al correr el código `STEGANALYSIS_E.py` con un total de 30,000 imágenes por limitantes de hardware, obtenemos el siguiente reporte:

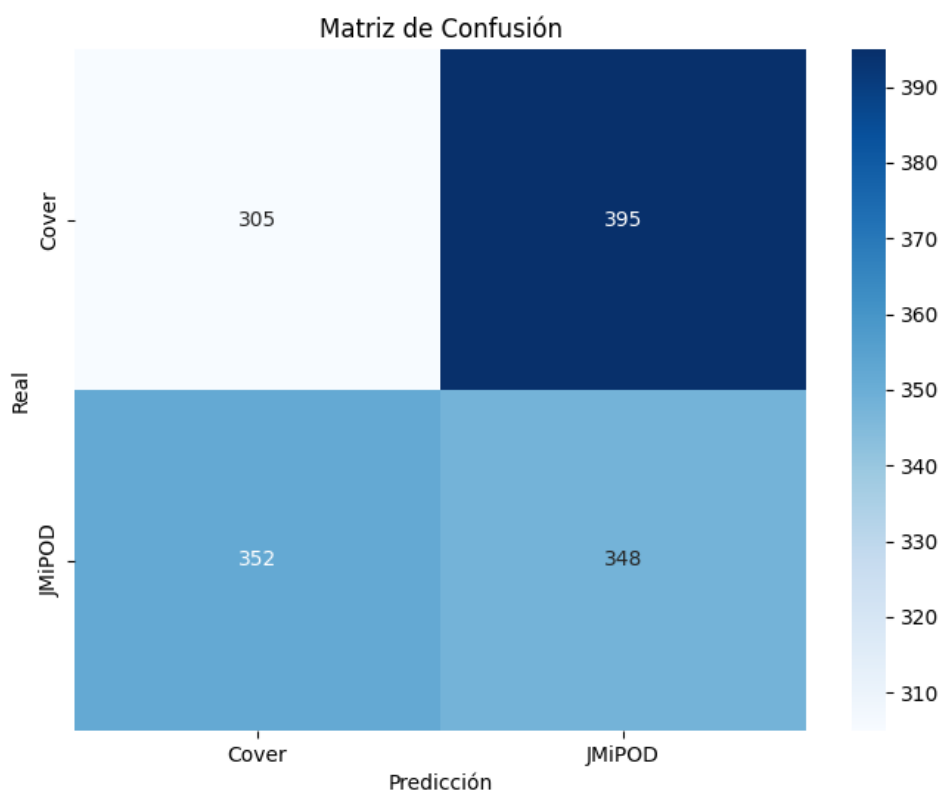
	precision	recall	f1-score	support
Cover	0.00	0.00	0.00	2500
JMiPOD	0.50	1.00	0.67	2500
accuracy			0.50	5000
macro_avg	0.25	0.50	0.33	5000
weighted_avg	0.25	0.50	0.33	5000

Estos resultados se dieron del mejor modelo que se dio en 13 épocas de 20 totales, lo que nos da a entender que el modelo esta **adivinando** siempre JMiPOD, por lo tanto, el modelo no esta aprendiendo a reconocer las imágenes y **siempre** asigna la etiqueta de JMiPOD.

Además de arrojar la siguiente matriz de confusión con la implementación del VGG16:



Mientras que sin utilizar el VGG16 y con otras configuraciones lograba llegar a tener diferentes resultados en la matriz de confusión:



De la que podemos interpretar que el modelo tenía muy bajo desempeño en general, con una confusión bastante significativa, y con un rango F1-score pobre, sin embargo, es diferente a lo encontrado en las siguientes iteraciones.

4.- Discusión

La teoría detrás de mi proyecto indica, que la combinación de el bloque SRM mas el pool L2, nos ayuda a destacar y fortalecer las señales de alta frecuencia, facilitando la separación de clases. Con ayuda del transfer learning, sobre VGG16 acelera la identificación y mejora la captura de patrones complejos respecto a una CNN entrenada desde cero lo que nos ayuda a optimizar el proceso.

Sin embargo, la limitación de utilizar solo 3 Kernels, SRM (vs 30 Originales) y recorte a 256x256 introduce la búsqueda de equilibrio entre detalles para analizar y el coste de hardware.

Pero en la realidad, mi proyecto sigue sin poder clasificar correctamente entre estos dos tipos de imágenes, ni con el modelo anterior al VGG16, ni después de su implementación,

igualmente puede deberse a una cantidad baja de épocas en el entrenamiento del modelo ya que solo se utilizaron como máximo 20 pero llegando a los mismos resultados.

5.- Conclusiones y trabajo futuro

Se deberían seguir los resultados de mas estados del arte que utilizan diferentes implementaciones de redes neuronales, para ver cuales son los resultados con una red entrenada desde cero con parámetros similares.

Otro de los problemas que considero importante es aumentar el tamaño del banco de kernels de 3 a 30, además de intentar utilizar diferentes backbones que sean mas ligeros como EfficientNet o MobileNet (que también fue recomendado) para despliegue en dispositivos limitados.

Se trato de diseñar un pipeline de detección de esteganografía en imágenes que combinaba un bloque fijo de filtros de alta frecuencia inspirados en el banco de SRM, la operacion del pooling y un clasificador denso final, lo que teóricamente debió mejorar el aprendizaje del modelo para poder diferenciar entre imágenes cover y JMiPOD

Referencias Bibliográficas:

- [1] Tabares-Soto et al., “Deep Learning Applied to Steganalysis of Digital Images: A Systematic Review,” *IEEE Access*, 2019.
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8718661>
- [2] Johnson & Katzenbeisser, “A Survey of Steganographic Techniques,” *Proc. of SPIE*, 2015.
- [3] Qian et al., “Convolutional Neural Networks for Steganalysis in Spatial Domain,” *IEEE TIFS*, 2015.
- [4] Xu et al., “Structural Coding of CNN for JPEG Steganalysis,” *WIFS*, 2017.
- [5] Fridrich & Kodovský, “Rich Models for Steganalysis,” *IEEE Trans. Information Forensics and Security*, 2012.
- [6] Yedroudj, M., Comby, F., & Chaumont, M. (2016). Yedroudj-Net: An efficient CNN for spatial steganalysis. En <https://www.researchgate.net/>. Recuperado 29 de abril de 2025, de https://www.researchgate.net/publication/327811038_Yedroudj-Net_An_Efficient_CNN_for_Spatial_Steganalysis